

Raising time-F1 and content-F1 on OmniPro Online

A diagnosis of the Qwen2.5-Omni fork, and what to do next

Threshold-fit working notes

1 September 2026

The short answer. The audio/video interleaving is *not* the problem — three separate checks clear it. The problem is upstream of the gate: the ICL control DSL that works on Qwen3-VL-8B **does not transfer** to Qwen2.5-Omni-7B. The controller generates a mean of **21.7 tokens per tick against a 300-token budget**, stopping on its own before the schema is finished; `fps` appears in **12 of 78,828** ticks and `next_check_s` in **8**. The gate is thresholding a boolean produced by a generation step that is not working, which is why $AUC \approx 0.5$ and why no threshold helped. That is a *fixable upstream cause*, not a property of the task.

1 Where we actually are

Stage 3, arm 1 (fitted per-task thresholds), 2,512 of 2,700 samples banked, judge offline so content metrics are WITHHELD.

stratum	n	n_{gt}	n_{emit}	t-P / t-R	time-F1
audio required	1643	4916	16185	0.155 / 0.510	0.2375
audio not required	868	3511	12866	0.164 / 0.601	0.2578
gross	2511	8427	29051	0.159 / 0.548	0.2464

Two facts frame everything below.

(1) **The system emits 3.4× more often than there are events** (29,051 emissions against 8,427 ground-truth triggers). Precision, not recall, is the binding constraint.

(2) **The gate is nearly exhausted.** §2.5 measured precision as *flat* across the whole threshold grid. With p pinned at 0.159, $F_1 = 2pr/(p+r)$ climbs monotonically as $r \rightarrow 1$ to a ceiling of $2p/(p+1) = \mathbf{0.274}$. We are at 0.246. Every remaining gate adjustment in existence is worth ≤ 0.028 — *inside* the 0.03 noise band — and collecting it means emitting even more. Note the corollary, which is counter-intuitive: cutting the 3.4× over-emission would *lower* F_1 , because precision will not rise to compensate.

1.1 Two different failure modes, not one

	task	gt/vid	emit/vid	time-F1
A. over-emitting	sequential_step_instruction	5.65	17.24	0.3778
	realtime_state_monitor	4.43	22.43	0.2431
	cumulative_counting	4.76	11.23	0.2337
	event_narration	4.37	19.22	0.2285
	dedup_counting	4.04	16.07	0.2097
	semantic_condition_alert	3.26	14.30	0.2071
B. right count, wrong moment	explicit_target_grounding	1.10	1.05	0.1065
	snapshot_counting	1.00	1.00	0.0674
	instant_event_alert	1.31	1.00	0.0604

The decisive row is `snapshot_counting`. Exactly 1.00 ground-truth event per video, exactly 1.00 emission per video — the emission *count is perfect* — and time-F1 is 0.0674. The one emission lands within ± 3 s of the truth roughly 7% of the time. Nothing about *how often* to speak is wrong there. *When* is wrong. No gate parameter can repair that.

Only `instant_event_alert` is genuinely refractory-capped: a 600s refractory against 1.31 events per video caps recall at ≈ 0.76 by construction. The other two group-B tasks emit the right number and mis-time it.

2 Five hypotheses, tested

hypothesis	verdict	evidence
TMRoPE positions are wrong	× cleared	numerically validated against <code>get_rope_index</code> (FORK §3); a 2s / 50-token audio chunk from anchor 8 yields exactly 8...57
Position clock outruns trained RoPE range	× cleared	max position reached $\approx 10,300$ vs <code>max_position_embeddings</code> = 32,768; the loud guard fired 0 times in 2,500+ videos; eviction never fired
Frozen perception (the parent's defect #1)	× cleared	median 110 distinct seen per video over 187 ticks; 0 of 532 videos emitted a single frozen description
Audio is not actually reaching the model	× cleared	<code>use_audio=True</code> on 2,514/2,514 , including all 1,645 <code>audio_dependency=required</code>
The ICL DSL does not transfer	✓ confirmed	§3

3 What is actually broken

3.1 The controller stops generating after ~22 tokens

measurement (78,828 controller ticks, 4 lanes)	value
<code>controller_max_tokens</code> budget	300
mean tokens actually generated per tick	21.7 (max 108)
ticks emitting seen	100%
ticks emitting answer	51%
ticks emitting <code>event_time_s</code>	41%
ticks emitting <code>fps</code>	12 (0.015%)
ticks emitting <code>next_check_s</code>	8 (0.010%)
ticks emitting <code>question_for_next</code>	21 (0.027%)
probe argmax is literally <code>true/false</code>	84.5%
probe argmax agrees with the parsed JSON	77%

The budget is 300 and the model uses 22. It is not being truncated — it **chooses to stop**, mid-schema. Representative **seen** payloads from the live logs:

```
"1 prices are ...' \n * Do not other numbers" • "{a single CJK character}" • "0\n{"
```

with `answer` fields reading `"{"` and `"1.0,"`. The JSON parses — 0% unparseable — so nothing raises an error. This is the project's signature failure mode: *a finished run and a number you could put in a table*.

Why this explains $AUC \approx 0.5$. `have_enough_info` is the boolean the gate thresholds. It is produced by the same degenerate generation step. A threshold on a boolean that the model is not really computing cannot rank event-adjacent ticks above quiet ones — so the four independent measurements of “no threshold helps” were all measuring one upstream defect, not nine hard tasks.

3.2 The self-pacing loop is dead

`fps` and `next_check_s` are how the ICL controller was supposed to *schedule itself* — look more often when something is happening, less when it is not. They appear in 20 ticks out of 78,828. The system is running at a fixed cadence with its adaptive machinery present in the prompt and absent from the output. This is the parent branch’s defect #7 (“fields simply never appeared”) recurring in a new form.

3.3 A verified structural divergence: the modality delimiters

The reference processor builds an interleaved A/V stream as

```
<vision_bos><audio_bos> [video chunk j][audio chunk j]* <audio_eos><vision_eos>
```

Our *ordering* matches it: video frames for $[t, t+2)$, then the 2s audio chunk for the same window. But **none of the four delimiter tokens appears anywhere in `async_omni_v2/`**. The model is fed an interleaved stream with no marker saying an interleaved stream has begun.

This is a real divergence from training conditions, verified from the reference source. It is **not** established as a *cause* — no ablation has been run. It is cheap to test and belongs high on the list precisely because it is cheap, not because it is proven.

4 Why the original ICL branch appears to score so much higher

The number is `joint_f1 0.206`, recorded in `EXPERIENCE.md` as “roughly MiniCPM-o-4.5-level”. Its own parenthetical is the important part: “*27-video, 9-task eval, older scoring.*”

`LEARNINGS.md §6b` then documents what “older scoring” meant — six defects, all in the scorer:

#	defect	effect on
13	LLM judge silently fell back to word overlap on any exception, while logging “google-genai judge active”	every j
17	state matched by substring , not equality	“RSM
15	<code>semantic_condition_alert</code> LLM-judged; upstream scores it <code>time_only</code>	wrong
14	<code>judge_cache.json</code> clobbered by 4 concurrent writers	24 h of
16	<code>_extract_count</code> did not strip clock times: “At 01:23, count is 5” = 1	correct

Defect #13 was caught by re-scoring in a bare shell **with no API key at all**, which reproduced the shipped `content_acc` to *four decimals*. A real judge and no judge cannot agree exactly — so no judge had ever run.

Aligning to the benchmark’s real scorer then found `realtime_state_monitor` at **0.000, 296 of 296 wrong**: it emitted sentences where the benchmark compares the whole payload to `state_to` by equality. That fix (emit the bare label) is commit `ab49ce9`, and it *is* present in this fork’s prompts.

And the deepest one, `LEARNINGS §2`: a null model using **only the timestamp**, with zero pixel access, beat `p_hit` on 3 of 4 tasks; `p_hit` correlated with video time at $|r|$ up to **0.93**. In the repo’s own words: “*The system was scoring worse than a clock.*”

Conclusion. 27 videos against our 2,700, a judge that was never running, and a control signal that was largely a clock. The 0.206 is not a target to chase. What *is* real in that branch are its **mechanisms**: hysteresis with re-arm (**hyst2b**), the per-tick **seen** step, word-overlap dedup within a true stretch, and the “already reported” history. Those survive the audit; the score does not.

5 What to do, in priority order

Ordered by (evidence it will help) $\times$ (cheapness), not by ambition.

Tier 1 — attacks the confirmed root cause

1. **Constrain the decode to the schema.** The model stops mid-JSON at 22 tokens. Guided/constrained decoding (a grammar or a logit-processor mask over the schema) makes the whole DSL structurally unskippable. This alone restores **answer** on the 49% of ticks that currently lack it and **event_time_s** on the 59% that lack it. *Cost: hours. Expected effect: large and broad.*
2. **Shrink the schema for this model.** Qwen2.5-Omni-7B is a weaker structured generator than Qwen3-VL-8B. Two fields (**have_enough_info**, **event_time_s**) may outperform seven that it never finishes. Test the reduced schema against the full one at equal budget. *Cost: hours.*
3. **Use **event_time_s** — the gate currently ignores it.** The model already names an onset time on 41% of ticks, and the emission is timestamped at the *crossing tick* instead. Emitting at the model’s stated time is a different, possibly much better, timing signal that costs **no GPU** to evaluate: it is already in the banked logs. *Do this first — it is free.*

Tier 2 — cheap, plausible, unproven

4. **Add the four modality delimiters** (§3.3) and A/B one task. Cheap, and it restores a training-time invariant we are currently violating.
5. **Detrend **p_hit** within each video.** The parent measured $|r|$ up to 0.93 between **p_hit** and wall-clock. If that holds here, most of the score is a clock; z-scoring or differencing within a video could expose the residual. **Free** — offline, from banked tick logs.
6. **Gate on the derivative, not the level.** Events are *changes*. A rising-edge gate on a level signal fires wherever the level is high. Also free to test offline.

Tier 3 — known-bounded gains

7. **Refit mode and refractory** (never refitted; inherited from the vision-only model). The three gate knobs are coupled, so this is not optional in principle — but the ceiling analysis says it repairs *one* task (**instant_event_alert**), not three.
8. **Do not chase the emission budget.** Matching emissions to event count would *reduce* F_1 while precision stays flat. Revisit only after Tier 1 makes the score informative.

Content-F1 specifically

9. **Restore a judge.** **content_acc** and **joint_f1** are WITHHELD — never guessed — and will stay so until one is reachable. The true value lies between the lower bound 0.038 and the timing ceiling 0.246; nothing narrows that without a judge. This blocks any comparison with MiniCPM-o 4.5 (0.209 *joint*-F1, not time-F1, and judged by a different model on a different subset).

10. **Fix answer degeneracy** — same root cause as item 1. An **answer** of "{" scores zero content no matter how good the timing is, so *every* joint-F1 point is currently gated behind Tier 1.
11. **Emit the benchmark's token form, not prose**, per task. Already done for `realtime_state_monitor`; audit the remaining constrained-format tasks the same way.

6 The honest bottom line

The threshold-fit study's negative result stands and is well-evidenced: *given this control signal*, no per-task threshold recovers information the signal does not carry, and the fit does not overfit (fit-disjoint bias +0.0007 gross). What the study could not see from inside is that **the signal itself is the artefact of a broken generation step**. The right next move is not a better gate. It is to make the controller finish its sentence.

Two caveats on this document. (i) The DSL-collapse diagnosis is measured; the claim that fixing it raises F_1 is a *hypothesis*, and item 3 is the free experiment that tests it before any GPU is spent. (ii) The audit of the parent branch's 0.206 is read from this repository's own post-mortems; that branch has not been re-run here.